6.100L 习题课 9

提醒事项

MQ 9 在 11 月 21 日星期一

过去两周的复习 ...

面向对象编程（OOP）

对象允许你在 Python 中存储数据

Python 中的一切都是对象

类定义了一种对象类型

到目前为止，在课堂上我们已经看到了以下内置类：int、float、string、list、tuples、dictionaries

对象是其类的实例

例如：3，“hello”，[1,2,3]都是类的实例

面向对象编程的优点：

允许您将数据捆绑成包

减少代码的复杂性，便于代码重用

允许您分别实现和测试行为每个类

类

一种使用内置数据类型作为构建模块来创建你自己的数据类型的方法

现实生活中的例子：电梯、员工、商店排队、松饼堆...

属性是属于类的数据与过程

数据属性：构成类的对象

方法/过程属性：仅与该类一起工作的函数

self：

指被调用方法的对象

定义方法时始终是第一个参数

不在类定义之外使用

创建类：

定义类名

class definition

name/type

class parent

```
class Coordinate(object):
    #define attributes here
```

定义类属性

定义如何使用 __init__ 方法创建类的实例

```
class Coordinate(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

what data initializes a Coordinate object

special method to create an instance — is double underscore

parameter to refer to an instance of the class

two data attributes for every Coordinate object

定义其他方法，这些方法不需要以 __ 开头

这些方法只适用于这个类。

```
class Coordinate(object):
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def distance(self, other):
        x_diff_sq = (self.x-other.x)**2
        y_diff_sq = (self.y-other.y)**2
        return (x_diff_sq + y_diff_sq)**0.5
```

use it to refer to any instance

another parameter to method

dot notation to access data

<u>使用类：</u>

首先创建一个类的新的实例

```
# 使用上面示例的 Coordinate 类

c1 = Coordinate(1,1)

c2 = Coordinate(2,1)
```

对实例执行操作，使用类方法

```
c1.distance(c2)
```

In order to be able to call print on an instance of your class you need to define the __str__ function.

there are a few more special operators (__len__, __eq__ etc… → look at lecture slides for details on these)

您可以使用 isinstance() 来检查一个实例是否是一个类的对象。

通常情况下类定义了所有类实例共有的表示和方法；而对象是类的特定实例

通常情况下您希望隐藏类的内部表示，以防止对抗性攻击和错误。

内部表示指的是您在 __init__ 方法中的内容

类变量：所有类的成员共享

在__init__方法外部定义

获取器和设置器方法：

- 获取器和设置器应在类外部使用，以访问数据属性。
- 这种方式访问属性是更好的风格，使代码更易于维护，并

有助于防止错误

继承与层次结构：

为什么使用继承？

- 允许你扩展一个类以获得新/不同的功能。
- 代码复用
- 共性在父类中明确，差异在子类中明确

结构及工作原理：

- 你通过定义两个类之间的关系来创建继承关系

进入类定义中的参数。

- Define parent class

```
class CreditCard(object):
```

Define child class:

```
class RewardCard(CreditCard):
```

子类继承了父类所有的方法。

- 我们可以在子类中定义新方法来扩展行为
- 我们可以重新定义父类中定义的方法来修改行为
- 当你在一个类的实例上调用一个方法：解释器试图查找

在类级别上使用方法，然后检查父类。

  - 这意味着如果定义了两个同名的函数，则

  子类中的方法优先级更高。

MIT 开放课程 ware
https://ocw.mit.edu

6.100L Python 语言计算机科学导论与编程

2022年秋季

如需了解引用这些材料或我们的使用条款，请访问：https://ocw.mit.edu/terms